Algorithms assignment 2

Thobias Høivik, Holly Storøy

For November 13, 2026

Appologies for the tangents and resulting length of this write-up.

Problem (2 Problem 5)**.** Given 7 jobs with corresponding processing times,

Job j	1	2	3	4	5	6	7
Processing Time p_j	4	7	2	9	12	5	3

The jobs will be processed on 3 identical machines without interruption. All jobs are avliable at time 0. (Model on slide 13, Chapter 2)

a) Use the local search algorithm where we have all jobs on machine 1 in the beginning (in the same order as they are given). When you move a job, move it to the machine it can start earliest. I there is a tie, use the machine with the lowest number. What is the result, both time when the last job finish and the actual schedule?
b) Use the Greedy algorithm on the same data. When have all jobs finished?
c) Use the Longest processing time rule. When have all jobs finished?
d) Give a lower bound on the best solution and give a short explanation?
e) Is it possible to achieve the answer in d) (Explain)?

This is the classical identical parallel-machine makespan minimization problem, denoted

$$P_3 \| \, C_{\max}.$$

We want to find a schedule minimizing

$$C_{\max} = \max_{i \in [3]} L_i,$$

where

$$L_i := \sum_{j \text{ assigned to machine } i} p_j$$

is the total processing time on machine i.

Solution to (a). The initial schedule gives all seven jobs to machine 1 in the described order by

$$M_1 = \underbrace{1}_{4} \quad \underbrace{1}_{4} \quad \underbrace{2}_{7} \quad \underbrace{3}_{2} \quad \underbrace{4}_{9} \quad \underbrace{5}_{12} \quad \underbrace{6}_{5} \quad \underbrace{7}_{3}.$$

Note: This accidentally looks a lot like fractions, but I am just illustrating the processing times p_j.

We get initial loads

$$(L_1, L_2, L_3) = (42, 0, 0)$$

with $C_{\max} = 42$.

If M_j is the unique machine for which $L_j = C_{\max}$ we find that it is advantageous to swap if

$$L_j + c < L_i,$$

or equivalently

$$c < L_i - L_j.$$

We perform the following procedure.

Require: Jobs J with processing times p_j, machines $M_1, \dots, M_m$
Ensure: A locally optimal schedule

```
Assign all jobs to M_1 in their given order
L_i ← Σ_{j∈M_i} p_j for each i ∈ {1, ..., m}
while true do
    i ← arg max_k L_k
    ℓ ← last job on M_i
    j ← arg min_{k≠i} L_k
    if L_j + p_ℓ ≥ L_i then
        return (M_1, ..., M_m)
    end if
    Remove ℓ from M_i
    Append ℓ to M_j
    L_i ← L_i − p_ℓ
    L_j ← L_j + p_ℓ
end while
```

So here we begin with

$$(42, 0, 0).$$

The job which finishes last is $j = 7$ so we get

$$(42, 0, 0) \mapsto (39, 3, 0).$$

Now M_1 still is the machine with maximum load so we take $j = 6$, and move

$$(39, 3, 0) \mapsto (34, 3, 5).$$

Repeat to get

$$(34, 3, 5) \mapsto (22, 15, 5) \mapsto (13, 15, 14).$$

Now we look at M_1 again and see that for $j = 3$ we have $p_j = 3 < L_3 = 14$ so the algorithm halts with

$$C_{\max} = 15 \leq 2C^*_{\max} = 14 \cdot 2 = 28$$

as expected.

Running time is $O(nm)$.

■

Solution to (b). We perform

Require: Jobs with processing times $p_1, \dots, p_n$ and m machines

1: $L_i \leftarrow 0$ for all $i = 1, \dots, m$
2: **for** $j = 1, \dots, n$ **do**
3: $\quad i^* \leftarrow \arg\min_i L_i$
4: $\quad$ Assign job j to machine M_{i^*}
5: $\quad L_{i^*} \leftarrow L_{i^*} + p_j$
6: **end for**
7: **return** $\max_i L_i$

We begin at $j = 1$ and since all M_i have $L_i = 0$ we get L_i's

$$(4, 0, 0).$$

Next

$$(4, 7, 0),$$

then

$$(4, 7, 2).$$

Once again L_3 is the minimum so next pass gives

$$(4, 7, 11).$$

Now L_1 is smallest so we get

$$(16, 7, 11).$$

Now L_2 is smallest so we get

$$(16, 12, 11).$$

Now L_3 is smallest so we get

$$(16, 12, 14).$$

We obtain $C_{\max} = 16$ with the algorithm running in time $O(nm)$. ■

Solution to (c). We perform

Require: Jobs J with processing times p_j, m identical machines

1: Sort jobs by nonincreasing processing time
2: $L_i \leftarrow 0$ for all $i = 1, \dots, m$
3: **for** each job j in sorted order **do**
4: $\quad i^* \leftarrow \arg\min_i L_i$
5: $\quad$ Assign job j to machine M_{i^*}
6: $\quad L_{i^*} \leftarrow L_{i^*} + p_j$
7: **end for**

8: **return** $\max_i L_i$

Performiing the algorithm we get

$$\begin{aligned}(12,0,0) &\mapsto (12,9,0)\\ &\mapsto (12,9,7)\\ &\mapsto (12,9,12)\\ &\mapsto (12,13,12)\\ &\mapsto (15,13,12)\\ &\mapsto (15,13,12)\end{aligned}$$

attaining a maximum of 15.

Running time is $O(n \log n + mn)$ since we sort first. ■

Solutioin and proof sketch for (d). Let $C^*_{\max}$ denote the optimal makespan, and let L_i be the total processing time assigned to machine M_i.

Since every job must be processed, we have

$$\sum_{i=1}^{m} L_i = \sum_{j=1}^{n} p_j.$$

Moreover, since each machine processes jobs sequentially,

$$C^*_{\max} \geq \max_i L_i \geq \frac{1}{m}\sum_{i=1}^{m} L_i = \frac{1}{m}\sum_{j=1}^{n} p_j.$$

We also necessarily have

$$C^*_{\max} \geq \max_j p_j,$$

since every job must finish before the schedule is complete.

Combining we get

$$C^*_{\max} \geq \max\left\{\frac{1}{m}\sum_{j=1}^{n} p_j,\ \max_j p_j\right\}.$$

■

Solution to (e). Consider the following schedule,

$$\begin{aligned}M_1 &: \{5,3\}, L_1 = 12+2 = 14\\ M_2 &: \{4,6\}, L_2 = 9+5 = 14,\\ M_3 &: \{2,1,7\} L_3 = 7+4+3 = 14.\end{aligned}$$

This is a feasible schedule with

$$C_{\max} = \max\{14, 14, 14\} = 14 = C^*_{\max}.$$

■

Problem (2, Problem 6)**.** The adjacency matrix of a graph on 6 vertices is given below. The triangle inequality holds for the whole graph.

$$\begin{bmatrix} 0 & 4 & 5 & 10 & 11 & 7 \\ 4 & 0 & 9 & 6 & 7 & 9 \\ 5 & 9 & 0 & 9 & 6 & 2 \\ 10 & 6 & 9 & 0 & 3 & 9 \\ 11 & 7 & 6 & 3 & 0 & 6 \\ 7 & 9 & 2 & 9 & 6 & 0 \end{bmatrix}$$

a) Verify that the triangle inquality holds when going from vertex 1 to 5.
b) Run the nearest addition algorithm on the graph. If there are more than one candidate to add in a step, use the vertex with the smallest number. It is sufficient to have fours nodes in the tour (skipping the last two iterations through the loop).
c) Run the Double tree algorithm on the graph. If there are more than one minimum spanning tree, you should run it on every tree.
d) Run Christofides' algorithm on the graph. If there are more than one minimum spanning tree, you should run it on every tree.

Let $G = ([6], E) \cong K_6$ be the complete graph (undirected) on 6 vertices with edge weights given in the adjacency matrix in the problem description.

Denote the weights c_{ij}. I.e. $c_{12} = 4,\ c_{15} = 11$ and so forth.

Solution for (a). We have $c_{15} = 11$. The possibilities are

$$\begin{aligned} c_{12} + c_{25} &= 4 + 7 = 11, \\ c_{13} + c_{35} &= 5 + 6 = 11, \\ c_{14} + c_{45} &= 10 + 3 = 13, \\ c_{16} + c_{65} &= 7 + 6 = 13. \end{aligned}$$

Thus

$$c_{15} = 11 \leq \min\{11, 11, 13, 13\} = 11$$

so we're good. ■

Solution to (b). We wish to find a permutation π of the vertices minimizing

$$c_{\pi(1)\pi(2)} + c_{\pi(2)\pi(3)} + \cdots + c_{\pi(6)\pi(1)}.$$

I.e. we want to find a minimum-weight Hamiltonian cycle. The additional assumption is that the weights satisfy the triangle inequality

$$c_{ij} \leq c_{ik} + c_{ki}, \ \forall i, j, k \in V = [6].$$

We begin with the two closest vertices and repeatedly add the vertex outside the tour that is closest to any vertex already in the tour.

When inserting a new vertex j between two consecutive vertices u, v, the increase in tour cost is

$$\Delta = c_{uj} + c_{jv} - c_{uv}.$$

Iteration 0: Init.

The smallest edge weight is

$$c_{36} = 2.$$

Thus, the initial tour is

$$T_0 = (3, 6, 3),$$

with cost 4.

Iteration 1.

For each unvisited vertex v, compute its distance to $S = \{3, 6\}$,

$$d(1, S) = 5$$
$$d(2, S) = 9$$
$$d(4, S) = 9$$
$$d(5, S) = 6.$$

Therefore, we choose vertex 1, whose nearest vertex is 3. We insert it into the tour to get

$$T_1 = (3, 1, 6, 3).$$

The cost is $w(T_1) = 14$.

Iteration 2.

Repeat to find $\arg\min_v d(v, S) = 2$ with weight 4. We have to places we can set it. If we insert it between 3 and 1 we get

$$\Delta_1 = c_{32} + c_{21} - c_{31} = 8.$$

If we choose 1 and 6 we get

$$\Delta_2 = 6.$$

Thus we choose it such that

$$T_2 = (3, 1, 2, 6, 3).$$

Now the cost is $w(T_2) = 20$. For the grader,

$$\text{weight is } \boxed{20}$$

■

Solution to (c). The four smallest edge weights are

$$\begin{aligned} c_{36} &= 2, \\ c_{45} &= 3, \\ c_{12} &= 4, \\ c_{13} &= 5. \end{aligned}$$

Kruskal's algorithm selects all four, giving two connected components in

$$A = \{1, 2, 3, 6\},\ B = \{4, 5\}.$$

The minimum weight of an edge joining these components is 6, attained by the edges

$$\{2, 4\},\ \{3, 5\},\ \{5, 6\}.$$

Formally, defininig

$$F = \{\{3, 6\},\ \{4, 5\},\ \{1, 2\},\ \{1, 3\}\},$$

the three MSTs are

$$\begin{aligned} T_A &= F \cup \{\{2, 4\}\} \\ T_B &= F \cup \{\{3, 5\}\} \\ T_C &= F \cup \{\{5, 6\}\}. \end{aligned}$$

Each has total weight

$$w(T_i) = 20.$$

Every edge in each MST is doubled, making every vertex degree even. Hence each resulting multigraph admits an Eulerian circuit.

We start at vertex 1 in each case, traversing tree branches in increasing order of vertex number.

For T_A, an Eulerian circuit is

$$W_A = (1, 2, 4, 5, 4, 2, 1, 3, 6, 3, 1).$$

Shortcutting repeated vertices yields

$$H_A = (1, 2, 4, 5, 3, 6, 1).$$

Its cost is

$$w(H_A) = 4 + 6 + 3 + 6 + 2 + 7 = 28.$$

For T_B take

$$W_B = (1, 2, 1, 3, 5, 4, 5, 3, 6, 3, 1).$$

Shortcutting gives

$$H_B = (1, 2, 3, 5, 4, 6, 1),$$

with cost

$$w(H_B) = 38.$$

Finally, for T_C, take

$$W_C = (1, 2, 1, 3, 6, 5, 4, 5, 6, 3, 1).$$

As before,

$$H_C = (1, 2, 3, 6, 5, 4, 1),$$

with

$$w(H_C) = 34.$$

All three satisfy

$$w(H_i) \leq 2w(T_i) = 40,$$

very cool. ■

Solution to (d). We add a minimum-weight perfect matching on the graph's odd-degree vertices. Consider the same three MSTs.

The edges of T_A form a path

$$5 - 4 - 2 - 1 - 3 - 6.$$

The only odd-degree vertices are endpoints $O_A = \{5, 6\}$, with only one possible perfect matching

$$P_A = \{\{5, 6\}\},\ w(P_A) = 6.$$

Adding this edge creates an Eulerian cycle, already Hamiltonian

$$H_A = (1, 2, 4, 5, 6, 3, 1),$$

with cost 26.

We repeat the procedure to produce

MST	Odd-degree vertices	Matching cost	Tour cost
T_A	$\{5, 6\}$	6	26
T_B	$\{2, 3, 4, 6\}$	8	28
T_C	$\{2, 4\}$	6	26

■

Problem (3, Problem 2)**.** I am too lazy to rewrite the entire table.

a) Run the dynamic programming algorithm (Chapter 3, slide 6) on the following kanpsack instance with $B = 10$. Show $A(i)$ twice (before and after removing the dominating pairs) for every iteration through the loop.
b) What is the maximum number of elements in $A(i)$, expressed in terms of B?
c) Let n be the number of items in the knapsack problem and let v_i be the value of item i. Furthermore let $V = \sum_{i=1}^{n} v_i$. What is the maximum of elements in $A(i)$ expressed in terms of V?

Recall that we want to maaximize

$$\sum_{i=1}^{6} y v_i x_i$$

such that

$$\sum_{i=1}^{6} s_i x_i \leq 10, \; x_i \in \{0, 1\}.$$

We define $A(i) \subseteq \mathbb{N}_0 \times \mathbb{N}_0$ where each pair $(s, v) \in A(i)$ represents a feasible choice of items from $[i]$ having total size s and value v.

We denote

$$A'(i) = A(i-1) \cup \{(s + s_i, v + v_i) \mid (s, v) \in A(i-1), \; s + s_i \leq B\}.$$

Solution to (a).

Iteration 1.

We begin with

$$A(0) = \{(0, 0)\}.$$

Taking or not taking the first item gives

$$A'(1) = \{(0, 0), \; (3, 5)\}.$$

Neither pair dominates the other so

$$A(1) = \{(0, 0), \; (3, 5)\}.$$

Iteration 2.

For each existing pair, we generate a new pair by adding $(2, 3)$,

$$(0, 0) + (2, 3) = (2, 3)$$
$$(3, 5) + (2, 3) = (5, 8).$$

Taking the union gives

$$A'(2) = \{(0,0), (2,3), (3,5), (5,8)\}.$$

Again, no pair is dominated so set

$$A(2) = A'(2).$$

Iteration 3.

We now generate the following pairs

$$\begin{aligned}(0,0) + (2,2) &= (2,2),\\ (2,3) + (2,2) &= (4,5),\\ (3,5) + (2,2) &= (5,7),\\ (5,8) + (2,2) &= (7,10).\end{aligned}$$

Before removing dominated pairs, we have

$$\begin{aligned}A'(3) = \{&(0,0), (2,3), (3,5), (5,8),\\ &(2,2), (4,5), (5,7), (7,10)\}.\end{aligned}$$

Observe thjat

$$\begin{aligned}(2,3) &\text{ dominates } (2,2),\\ (3,5) &\text{ dominates } (4,5),\\ (5,8) &\text{ dominates } (5,7).\end{aligned}$$

Prune to get

$$A(3) = \{(0,0), (2,3), (3,5), (5,8), (7,10)\}.$$

Iteration 4.

We generate

$$\begin{aligned}(0,0) + (4,5) &= (4,5)\\ (2,3) + (4,5) &= (6,8)\\ (3,5) + (4,5) &= (7,10)\\ (5,8) + (4,5) &= (9,13)\\ (7,10) + (4,5) &= (11,15).\end{aligned}$$

We get

$$\begin{aligned}A'(4) = \{&(0,0), (2,3), (3,5), (5,8),\\ &(4,5), (6,8), (7,10), (9,13)\}.\end{aligned}$$

Notice we do not include $(11,15)$ as $11 > B = 10$. Here

$$\begin{aligned}(3,5) &\text{ dominates } (4,5)\\ (5,8) &\text{ dominates } (6,8),\end{aligned}$$

so we are left with

$$A(4) = \{(0,0), (2,3), (3,5), (5,8), (7,10), (9,13)\}.$$

Iteration 5.

We generated

$$\begin{aligned}(0,0) + (3,4) &= (3,4)\\ (2,3) + (3,4) &= (5,7)\\ (3,5) + (3,4) &= (6,9)\\ (5,8) + (3,4) &= (8,12)\\ (7,10) + (3,4) &= (10,14)\\ (9,13) + (3,4) &= (12,17).\end{aligned}$$

We discard $(12, 17)$ and get

$$\begin{aligned}A'(i) = \{&(0,0), (2,3), (3,5), (5,8), (7,10), (9,13)\\ &(3,4), (5,7), (6,9), (8,12), (10,14)\}.\end{aligned}$$

Here

$$\begin{aligned}(3,5) &\text{ dominates } (3,4)\\ (5,8) &\text{ dominates } (5,7),\end{aligned}$$

leaving

$$\begin{aligned}A(5) = \{&(0,0), (2,3), (3,5), (5,8),\\ &(6,9), (7,10), (8,12), (9,13), (10,14)\}.\end{aligned}$$

Iteration 6.

We generate

$$\begin{aligned}(0,0) + (1,2) &= (1,2)\\ (2,3) + (1,2) &= (3,5)\\ (3,5) + (1,2) &= (4,7)\\ (5,8) + (1,2) &= (6,10)\\ (7,10) + (1,2) &= (8,12)\\ (6,9) + (1,2) &= (7,11)\\ (8,12) + (1,2) &= (9,14)\\ (9,13) + (1,2) &= (10,15)\\ (10,14) + (1,2) &= (11,16).\end{aligned}$$

From this we get

$$\begin{aligned}A'(6) = \{&(0,0), (2,3), (5,8), (7,10), (6,9), (8,12), (10,14),\\ &(1,2), (3,5), (4,7), (6,10), (8,12), (7,11), (9,14), (10,15)\}.\end{aligned}$$

Here

$$(6, 10) \text{ dominates } (7, 10)$$
$$(9, 14) \text{ dominates } (10, 14)$$
$$(6, 10) \text{ dominates } (6, 9).$$

We have

$$\begin{aligned} A(6) = \{&(0, 0), (1, 2), (2, 3), (3, 5), \\ &(4, 7), (5, 8), (6, 10), (7, 11), \\ &(8, 12), (9, 14), (10, 15)\}. \end{aligned}$$

Thus OPT $= 15$. One optimal selection consists of $1, 2, 4, 6$ giving $\sum s = 10$, and $\sum v = 15$. ■

Statement and proof of (b). A pair (s, v) dominates (s', v'), writing $(s', v') \prec (s, v)$, if

$$s \leq s', \text{ and } v \geq v'$$

with at least one inequality strict.

We claim that $|A(i)| \leq B + 1$.

Every pair $(s, v) \in A(i)$ represents a feasible selection of items. Since item sizes are nonegative integers and the knapsack has capacity B, we have

$$s \in \{0, 1, \dots, B\}.$$

Suppose, for a contradiction, that

$$|A(i)| > B + 1.$$

By the pigeonhole principle, there must exist two distinct pairs

$$(s, v_1), (s, v_2) \in A(i)$$

with the same size s.

Since the pairs are distinct, $v_1 \neq v_2$, assume, without loss of generality, that $v_1 > v_2$. Then

$$(s, v_1) \succ (s, v_2),$$

contradicting the definition $A(i)$ as a set of nondominated pairs.

Therefore

$$|A(i)| \leq B + 1.$$

■

Statement and proof of (c). We claim that

$$|A(i)| \leq V + 1.$$

Let

$$V = \sum_{j=1}^{n} v_j.$$

Since all item values are nonnegative integers, every pair $(s, v) \in A(i)$ satisfies

$$v \in \{0, 1, \dots, V\}.$$

Suppose, once again, for a contradiction, that

$$|A(i)| > V + 1.$$

By the pigeonhole principle, there exist distinct

$$(s_1, v), (s_2, v) \in A(i)$$

with the same value v.

Without loss of generality, suppose $s_1 < s_2$. Then

$$(s_1, v) \succ (s_2, v),$$

again contradicting the fact that $A(i)$ contains only nondominated pairs.

Consequently,

$$|A(i)| \leq V + 1.$$

■

Problem (3, Problem 6)**.** Consider the problem of minimizing the sum of completion times on a single machine given the data below:

imagine the table here

Use the approximation algorithm based on the Shortest Remaining Processing Time for preemptive scheduling to find an approximate solution to the nonpreemptive problem.

Solution. At $t = 0$, only jobs $1, 2$ are avaliable and $p_2 = 5 < p_1 = 6$ so execuite job 2. At $t = 1$, job 3 arrives. Job 2 has 4 units left, while job 3 requires 3 so we interrupt job 2 to execute job 3.

Repeat as such to get

Time interval	executed job
$[0,1]$	2
$[1,2]$	3
$[2,3]$	4
$[3,5]$	3
$[5,7]$	6
$[7,11]$	2
$[11,15]$	5
$[15,21]$	1
$[21,23]$	7

Reading of the last time j appears we get completion times

$$C_1 = 21,\ C_2 = 11,\ C_3 = 5,$$
$$C_4 = 3,\ C_5 = 15,\ C_6 = 7,\ C_7 = 23.$$

Using the standard completion-order conversion we have a possible nonpreemptive schedule in

$$4 \to 3 \to 6 \to 2 \to 5 \to 1 \to 7$$

with completion times

$$(3, 6, 8, 13, 17, 23, 25)$$

with total time for this nonpreemptive schedule being

$$3 + 6 + 8 + 13 + 17 + 23 + 25 = 95.$$

■

Problem (4, Problem 2)**.**

a) Implement the randomized algorithm for MAX SAT where we set each x_i to true independently with probability $1/2$.
b) Assign values to $x_1, \dots, x_n$ by the derandomized version of the algorithm based

on methods of computational expectations to the following clauses

$$
\begin{aligned}
C_1 &= 3(x_1 \vee \overline{x_2} \vee x_3) \\
C_2 &= 4(x_1 \vee \overline{x_3}) \\
C_3 &= 1(\overline{x_1} \vee x_2 \vee \overline{x_5}) \\
C_4 &= 3(\overline{x_1} \vee x_3) \\
C_5 &= 2(x_2 \vee \overline{x_4}) \\
C_6 &= 7(\overline{x_2} \vee x_5 \vee x_9) \\
C_7 &= 2(x_3 \vee \overline{x_4}) \\
C_8 &= 3(\overline{x_1} \vee \overline{x_5} \vee x_{21}) \\
C_9 &= 5(x_5 \vee \overline{x_{12}}) \\
&\vdots
\end{aligned}
$$

The clauses $C_1, \dots, C_9$ are the only clauses containing the variables $x_1, \dots, x_5$.

Solution to (a). We implement the randomized MAX-SAT algorithm by independently assigning each variable the value True with probability $1/2$.

```python
import random

def randomized_max_sat(clauses):
    variables = {
        abs(lit)
        for weight, clause in clauses
        for lit in clause
    }

    assignment = {
        i: bool(random.getrandbits(1))
        for i in variables
    }

    score = sum(
        weight
        for weight, clause in clauses
        if any(
            assignment[abs(lit)] == (lit > 0)
            for lit in clause
        )
    )

    return assignment, score
```

Solution to (b). Let $Z = \sum_{j=1}^{9} w_j I_j$, where I_j indicates whether clause C_j is satisfied.

We assign $x_1, \dots, x_5$ sequentially using the method of conditional expectations. At each step, all unassigned variables remain independently true with probability $1/2$.

Initially, the expected satisfied weight is

$$\begin{aligned}\mathbb{E}[Z] &= \frac{7}{8}(3+1+7+3) + \frac{3}{4}(4+3+2+2+5) \\ &= \frac{97}{4}.\end{aligned}$$

Suppose $x_1 = a_1, \dots, x_{i-1} = a_{i-1}$ have already been assigned. Define

$$E_b^{(i)} = \mathbb{E}[Z \mid x_1 = a_1, \dots, x_{i-1} = a_{i-1}, x_i = b]$$

for $b \in \{0, 1\}$. We choose

$$a_i \in \arg\max_{b \in \{0,1\}} E_b^{(i)}.$$

For x_1, only clauses $C_1, \dots, C_4$ are affected. The remaining clauses contribute $31/2$ in expectation. Therefore,

$$\begin{aligned}E_0^{(1)} &= \frac{31}{2} + 3 \cdot \frac{3}{4} + 4 \cdot \frac{1}{2} + 1 + 3 = \frac{95}{4}, \\ E_1^{(1)} &= \frac{31}{2} + 3 + 4 + \frac{3}{4} + 3 \cdot \frac{1}{2} = \frac{99}{4}.\end{aligned}$$

Thus $a_1 = 1$.

Repeating this calculation for each variable, conditioned on the preceding choices, gives

Variable	$E_0^{(i)}$	$E_1^{(i)}$	a_i
x_1	$95/4$	$99/4$	1
x_2	$199/8$	$197/8$	0
x_3	$183/8$	$215/8$	1
x_4	$113/4$	$51/2$	0
x_5	$55/2$	29	1

Hence the algorithm produces

$$(a_1, a_2, a_3, a_4, a_5) = (1, 0, 1, 0, 1).$$

Under this assignment, all nine displayed clauses except C_3 are satisfied. Their total weight is 30, so the satisfied weight is

$$\boxed{Z = 30 - w_3 = 29}.$$

Finally, at each step the law of total expectation gives

$$\mathbb{E}[Z \mid a_1, \dots, a_{i-1}] = \frac{1}{2}E_0^{(i)} + \frac{1}{2}E_1^{(i)} \leq \max\{E_0^{(i)}, E_1^{(i)}\}.$$

Therefore, every choice preserves or increases the conditional expectation. In particular, the final satisfied weight is at least the initial expectation $97/4$. ■

Problem (4, Problem 3)**.** We have the following MAX SAT problem

$$C_1 = 4(x_1 \vee \overline{x_3}),\ C_2 = 3(\overline{x_1} \vee x_4),\ C_3 = 2(\overline{x_2} \vee x_4),\ C_4 = 1(\overline{x_2} \vee \overline{x_3} \vee x_4).$$

a) Find the ILP (IP) formulation of the problem above.
b) Find the LP formulation for the problem above.

Statement and proof sketch of (a/b). Intuitively, we'd want something like:

Maximize

$$\sum_{i=1}^{4} w_i \left(\bigoplus_{\ell \in C_i} \ell \right),$$

subject to

$$x_1, x_2, x_3, x_4 \in \{0, 1\},$$

but this isn't a valid ILP, of course.

So instead let $x_1, x_2, x_3, x_4 \in \{0, 1\}$ and introduce $y_1, y_2, y_3, y_4 \in \{0, 1\}$ where $y_i = 1$ indicates that C_i is satisfied.

We maximize

$$4y_1 + 3y_2 + 2y_3 + y_4,$$

subject to

$$\begin{aligned} x_1 + (1 - x_3) &\geq y_1, \\ (1 - x_1) + x_4 &\geq y_2, \\ (1 - x_2) + x_4 &\geq y_3, \\ (1 - x_2) + (1 - x_3) &\geq y_4. \end{aligned}$$

This is a valid ILP formulation and we claim that it corresponds one-to-one with the MAX SAT instance.

$\text{OPT}_{\textbf{ILP}} \geq \text{OPT}_{\textbf{MAXSAT}}$.

Let x^* be an optimal truth assignment for the MAX-SAT instance.

Construct an ILP solution by choosing the same values for the variables x_j and setting

$$y_i = \begin{cases} 1 & \text{if } C_i \text{ is satisfied by } x^* \\ 0 & \text{otherwise.} \end{cases}$$

We claim this solution is feasible.

Indeed, if C_i is satisfied, at least on literal evaluates to 1, and therefore

$$\sum_{i \in C_i} v(\ell) \geq 1 = y_i,$$

where we denote

$$v(\ell) = \begin{cases} x_j & \text{if } \ell = x_j \\ 1 - x_j & \text{if } \ell = \bar{x}_j. \end{cases}$$

Otherwise, $y_i = 0$ and the constraint holds because every literal evaluates to a nonnegative number.

Thus, the constructed solution is feasible, and its objective value is the total weight of clauses satisfied by x^*.

Consequently

$$\text{OPT}_{\text{ILP}} \geq \text{OPT}_{\text{MAXSAT}}.$$

$\text{OPT}_{\textbf{ILP}} \leq \text{OPT}_{\textbf{MAXSAT}}$.

Now let (x^*, y^*) be an optimal ILP solution.

Consider the truth assignment obtained by interpreting $x_j^* = 1$ as true and 0 as false.

For every clause C_i with $y_i^* = 1$, the corresponding constraint gives

$$\sum_{\ell \in C_i} v(\ell) \geq 1.$$

Sinc eterms are binary, at least one literal must evaluate to 1. Hence C_i is satisfied.

Therefore, the assignment satisfies at least all clauses for which $y^* = 1$. Since weights are positive, its total weight is at least

$$\sum_{i=1}^{4} w_i y_i^* = \text{OPT}_{\text{ILP}}.$$

But the optimal MAX-SAT assignment cannot have a smaller value than any particular assignment. Thus

$$\text{OPT}_{\text{ILp}} \leq \text{OPT}_{\text{MAXSAT}}.$$

For the LP relaxation we simply replace the constraints by contiuous bounds

$$x_i \in [0, 1], \; y_i \in [0, 1].$$

Asside.

Also, for fun, here is MAX-SAT encoded as a maximisation problem in a a quotient ring, because that's funny.

For the polynomial ring

$$\mathbb{Q}[x_1, x_2, x_3, x_4] \qquad \text{(nice and computable)},$$

notice that

$$x_i \in \{0, 1\} \Leftrightarrow x_i^2 - x_i = 0.$$

Define the ideal

$$I = (x_1^2 - x_1, \dots, x_4^2 - x_4)$$

and quotient to get

$$R = \mathbb{Q}[x_1, x_2, x_3, x_4]/I.$$

The affine variety $V(I)$ is

$$V(I) = \{0, 1\}^4.$$

By the Chinese remainder theorem,

$$R \cong \prod_{a \in \{0,1\}} \mathbb{Q} \cong \mathbb{Q}^{16}.$$

We want a polynomial s_i evaluating to 1 when C_i is satisfied.

For our clauses the following

$$\begin{aligned} s_1 &= 1 - (1 - x_1)x_3, \\ s_2 &= 1 - x_1(1 - x_4), \\ s_3 &= 1 - x_2(1 - x_4), \\ s_4 &= 1 - x_2x_3(1 - x_4) \end{aligned}$$

work.

For each s_i we have

$$s_i(a) = \begin{cases} 1 & \text{if } a \vDash C_i, \\ 0 & \text{otherwise.} \end{cases}$$

Setting $F = 4s_1 + 3s_2 + 2s_3 + s_4$ gives our objective function and we can reformulate the problem as

$$\max_{a \in V(I)} F(a).$$

Since all weights are integers we have

$$F(a) \in \{0, 1, \dots, 10\}.$$

So

$$F(a) \geq k$$

iff

$$\prod_{r=k}^{10} (F(a) - r) = 0,$$

letting us define an ideal

$$J_k = I + \left(\prod_{r=k} (F - r) \right),$$

giving way to

$$\mathrm{OPT}_{\mathrm{MAXSAT}} \geq k \iff V(J_k) \neq \emptyset \iff S/J_k \neq 0.$$

If wee consider the $\mathbb{Q}$-subalgebra generated by F,

$$\mathbb{Q}[F] \subseteq R,$$

we can, by the first isomorphism theorem, say that the evaluation homomorphism

$$\varphi : \mathbb{Q}[t] \to R,\ t \mapsto F,$$

gives

$$\mathbb{Q}[F] \cong \mathbb{Q}[t]/\ker \varphi.$$

This gives

$$\ker \varphi = \left(\prod_{9 \neq i = 3}^{10} (t - i) \right),$$

so

$$\mathbb{Q}[F] = \frac{\mathbb{Q}[t]}{\prod_{i \neq i = 3}^{10} (t - i)}.$$

Then the largest root is the optimal MAX-SAT value.

We have $R \cong \mathbb{Q}^{2^n}$ so this would probably be ridiculously inefficient for large MAX-SAT instances, but its way cooler than integer programming. ■

> **Problem** (5, Problem 1)**.** The graph below is 3-colorable. Use the algorithm that colors a 3-colorable graph with $O(\sqrt{n})$ colors, Chapter 6 to color the graph. Every time you have a choice between nodes, choose the lowered number one. Notice, it is important to understand the algorithm, not find the best coloring.

Solution. I believe we are meant to use Avi Widgerson's algorithm as it is a famous (promised-3-colorable) coloring algorithm giving $O(\sqrt{n})$ colors.

We remind ourselves how the algorithm goes.

Let $n = |V|$ throughout the algorithm. While $\Delta(G) > \sqrt{n}$, choose a vertex v of maximum degree. Color its induced neighborhood $G[N(v)]$ using two colors, give v a third color, and remove $N[v]$.

Once $\Delta(G) \leq \sqrt{n}$, greedily color the remaining graph.

Simple enough!

At iteration 1 we have $\Delta(G) = 4 > 3 = \sqrt{n}$. Both 1 and 6 have maximum degree. By the

tie-breaking rule we choose the first.

The neighborhood is

$$N(1) = \{3, 4, 6, 8\}.$$

The induced subgraph is the path 4–3–6–8.

We 2-color by

$$\begin{aligned} \text{Color 1}: &\quad \{3, 8\}, \\ \text{Color 2}: &\quad \{4, 6\}. \end{aligned}$$

Give vertex 1 color 3.

Remove

$$N[1] = \{1, 3, 4, 6, 8\}$$

to get

$$V' = \{2, 5, 7, 9\}.$$

The remaining edges are

$$E' = \{\{2, 7\}, \{2, 9\}, \{5, 9\}, \{7, 9\}\}.$$

Now $\Delta(G)$ is small enough that we go to phase 2.

We now greedily color in numerical order. Using Wigderson's formulation, the next available color is color 3.

Then

Vertex	Previosuly colored neighbors in G'	Assigned color
2	None	3
5	None	3
7	2 (color 3)	4
9	2,5,7 (colors 3,3,4)	5.

Thus the final coloring is

$$\begin{aligned} V_1 &= \{3, 8\}, \\ V_2 &= \{4, 6\}, \\ V_3 &= \{1, 2, 5\}, \\ V_4 &= \{7\}, \\ V_5 &= \{9\}. \end{aligned}$$

If we don't employ Wigderson's formulation allowing color 3 to be valid, since 1 had no neighbors among the remaining vertices, then we might end up with 7 colors instead.

A cool observation is that, while this algorithm runs in $O((n+m)\sqrt{n})$ (where $m = |E|$, also, running time can be improved) time with a guarantee of $O(\sqrt{n})$ colors, it was conjectured that k-colorability of 3-colorable graphs is NP-hard for all k, only that it was really hard to prove.

This algorithm gives some insight as to why, since for $k = O(\sqrt{n})$ we (given all reasonable

assumptions) cannot have hardness, making the difficulty of proving the difficulty harder for increasing k.

However, as Fei, Minzer, and Wang proved the 4-to-1 Games Conjecture back in september, we have that k-coloring of promised-3-colorable graphs is NP-hard $\forall k \geq 3$.

Very cool!

■

Problem. A vertex cover of a graph is a subset $C \subseteq V$ such that every edge has at least one endpoint in C, that is $\forall \{i, j\} \in E$ either $i \in C$or $j \in C$. We want to find a minimum vertex cover.

imagine the graph here

a) Formulate the problem above as an ILP.
b) Formulate the LP relaxation of the ILP.
c) Formulate the Dual version of the LP problem.
d) We have a large graph with 10.000 vertices. We solve the LP-problem and get the solution 500 (number of vertices in the cover). What can we say about the solution to the original problem in a)?

Solution with no proof this time to (a/b/c). We assert that the below formulation works.

Minimize

$$\sum_{v \in V} x_v,$$

subject to

$$x_u + x_v \geq 1, \ \forall \{u, v\} \in E,$$

and

$$x_v \in \{0, 1\}, \ \forall v \in V.$$

For the LP relaxation we simply allow

$$x_v \in [0, 1].$$

For the primal LP

$$\begin{aligned} \min c^T x \\ Ax \geq b \\ x \geq 0 \end{aligned}$$

we have that the dual is

$$\begin{aligned} \max b^T y \\ A^T y \leq c \\ y \geq 0 \end{aligned}$$

So we introduce one dual variable $y_e,\ \forall e \in E$.

Since the right-hand side of every primal edge constraint is 1, the dual objective is

$$\max \sum_{e \in E} y_e.$$

Meanwhile, each primal x_v appears in the constraints corresponding to edges with v. Consequently, the corresponding dual constraint is

$$\sum_{e \ni v} y_e \leq 1,$$

obtaining

$$\begin{aligned} \text{maximize} \quad & \sum_{e \in E} y_e \\ \text{subject to} \quad & \sum_{e \ni v} y_e \leq 1 \quad \forall v \in V, \\ & y_e \geq 0 \qquad\quad \forall e \in E. \end{aligned}$$

This encodes maximum fractional matching, representative of the dual relationship between minimal vertex covers and maximal matchings. Very cool! ■

Solution to (d). Let $\text{OPT}_{\text{LP}} = 500$. Since every feasible integer solution is also feasible in the relaxation we have

$$500 \leq \text{OPT}_{ILP}.$$

Helpful, but trivial.

Suppose x^* is an optimal fractional solutiion. Define

$$C = \{v \in V \mid x_v^* \geq \frac{1}{2}\}.$$

We claim that this is a vertex cover.

Indeed, for every $\{u, v\} \in E$, we have

$$x_u^* + x_v^* \geq 1.$$

Consequently, at least on of x_u^*, x_v^* must be at least $1/2$ so at least one endpoint belongs to C.

Moreover

$$\begin{aligned} |C| &= \sum_{v \in V} \mathbf{1}[x_v^* \geq \tfrac{1}{2}] \\ &\leq 2 \sum_{v \in V} x_v^* \\ &= 2 \cdot 500 = 1000. \end{aligned}$$

Thus we have a vertex cover of size at most 1000, letting us conclude

$$500 \leq \text{OPT}_{ILP} \leq 1000.$$

■